PCCST501 – COMPUTER NETWORKS

ASSIGNMENT 1

Name: Evan Tigi

Class: S5 CSA

Roll No: 29

MODULE 1 ASSIGNMENT

How the Internet Delivers a Web Page

This report explains the complete process that occurs when a user types www.instagram.com into a web browser and presses Enter, covering the TCP/IP protocol stack, DNS resolution, HTTP request/response, TCP reliability, application architecture, and the role of cookies.

1. The TCP/IP Protocol Stack

When Instagram is accessed, data travels down through five layers on the client (browser) side, across the network, and back up the same five layers on the server side.

TCP/IP Protocol Stack (used when loading instagram.com)

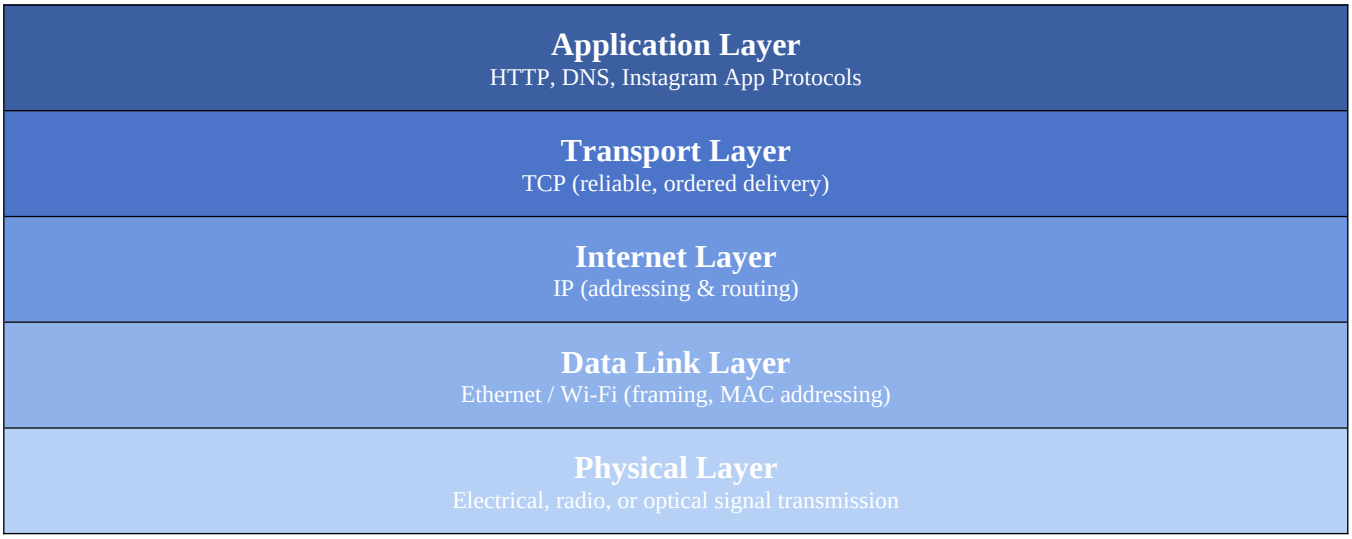


Figure 1: TCP/IP Protocol Stack for an Instagram request

- **Application Layer:** Generates the actual request the user cares about. HTTP builds the GET request for the Instagram page, and DNS resolves the domain name to an IP address.
- **Transport Layer:** TCP breaks the HTTP message into segments, numbers them, and guarantees reliable, in-order, error-checked delivery between the browser and Instagram's server.
- **Internet Layer:** IP wraps each TCP segment in a packet with source and destination IP addresses and routes it across intermediate networks/routers to reach Instagram's data centre.
- **Data Link Layer:** Organises IP packets into frames and adds physical (MAC) addressing so devices on the same local network (LAN/Wi-Fi) can identify each other.
- **Physical Layer:** Physically transmits the frame's bits as electrical, radio (Wi-Fi), or optical signals over the transmission medium connecting the devices.

2. Role of DNS in Resolving instagram.com

Computers route traffic using numeric IP addresses, not domain names, so DNS (Domain Name System) acts as the internet's phonebook. When the user enters instagram.com, the browser first checks its own cache and the operating system's cache for a recent match. If none is found, a DNS query is sent to a recursive resolver (usually run by the ISP), which in turn queries a root server, then a .com top-level-domain (TLD) server, and finally Instagram's authoritative name server. That server returns the IP address

bound to instagram.com, which the browser caches and uses to establish a connection to the correct server.

3. How HTTP Requests and Receives the Webpage

Once the browser has Instagram's IP address, it uses HTTP (almost always HTTPS, its encrypted form) to communicate with the web server. The browser sends an HTTP GET request for the page (e.g., GET / HTTP/1.1) along with headers such as the host name, accepted content types, cookies, and browser information. Instagram's server processes this request, retrieves the required feed data from its backend, and returns an HTTP response containing a status code (such as 200 OK), response headers, and the actual content — HTML, CSS, JavaScript, and image/video data — which the browser then renders as the visible page.

4. How TCP Provides Reliable Communication for HTTP

HTTP relies on TCP to guarantee that its request and response messages arrive completely and correctly. Before any HTTP data is sent, the browser and server perform a TCP three-way handshake (SYN, SYN-ACK, ACK) to establish a connection. TCP then splits the HTTP message into numbered segments, and the receiving side sends acknowledgements for each segment received. If a segment is lost or corrupted, TCP detects this (via missing acknowledgements or checksums) and retransmits it. TCP also reassembles segments in the correct order and manages flow/congestion control so data is not sent faster than the network or receiver can handle — ensuring the Instagram page loads with no missing or corrupted content.

5. Client–Server or Peer-to-Peer Architecture

Instagram follows a Client–Server architecture, not a peer-to-peer one. All user devices (clients) send requests to Instagram's centralized servers/data centres, which store user data, photos, and videos, and run the application logic. Clients never communicate directly with one another to exchange content; every interaction — viewing a post, sending a direct message, or liking a photo — is mediated by Instagram's servers. This design gives Instagram centralized control over content moderation, data storage, authentication, and scalability, which a decentralized peer-to-peer model would make far harder to achieve at Instagram's scale.

6. HTTP vs FTP – At Least Four Differences

Aspect	HTTP	FTP
Purpose	Transfers and renders web pages/resources	Transfers files between client and server
Connections used	Typically one connection for request/response	Two connections: control (commands) and data (file transfer)
Statefulness	Stateless by default (cookies used to add state)	Stateful — maintains a session throughout the transfer
Port used	Port 80 (HTTP) / 443 (HTTPS)	Port 21 (control) and Port 20 (data)
Typical use case	Loading the Instagram web page and its assets	Uploading/downloading large files to/from a server

7. How Cookies Improve User Experience

Because HTTP is stateless, cookies are small pieces of data that Instagram's server stores on the user's browser to remember information across multiple requests and visits. For example, after a user logs into Instagram once, a session/authentication cookie is stored on the device so the user stays logged in on subsequent visits instead of having to enter credentials every time. Cookies are also used to remember language preferences, keep track of items in a shopping cart on other e-commerce sites, and personalize the content feed based on prior activity — all of which make browsing faster and more convenient.

8. Flow Diagram: Browser to Web Server and Back

Flow: Browser ↔ Instagram Web Server

1. User types instagram.com and presses Enter
2. Browser checks cache; if empty, sends DNS query
3. DNS Resolver returns IP address of Instagram's server
4. Browser opens TCP connection to server (TCP 3-way handshake)
5. Browser sends HTTP GET request over TCP (with cookies, if any)
6. Instagram's web server processes request, queries backend/DB
7. Server sends back HTTP response (HTML, CSS, JS, images)
8. TCP ensures all packets arrive reliably & in order
9. Browser renders the Instagram feed page for the user

Figure 2: End-to-end communication flow for loading Instagram

9. Conclusion

Loading a single Instagram page is the result of several application-layer protocols working together seamlessly within seconds. DNS translates a human-friendly domain name into a machine-usable IP address, HTTP structures the request and response so the browser and server understand each other, and TCP guarantees that the exchanged data is complete, ordered, and error-free, all carried by IP across the underlying network. Supporting features such as cookies add statefulness on top of an inherently stateless protocol, letting services like Instagram remember logged-in users and personalize their experience. Without these application-layer protocols operating in harmony, everyday tasks such as browsing a social media feed, sending an email, or streaming a video would not be possible. Understanding this layered process highlights why the TCP/IP model remains the backbone of how billions of devices communicate over the internet every day, and why even a small delay or failure in DNS resolution, TCP handshakes, or HTTP processing can be felt immediately by end users as a slow or broken page load.